Code Samples (Some may hit)

Bootstrapping Code

```
import numpy as np

def bootstrap_confidence_interval(data, iterations=1000):
    """
    Bootstrap the 95% confidence interval for the mean of the data.
Parameters:
    - data: An array of data    - iterations: The number of bootstrap
samples to generate        Returns:
    - A tuple representing the lower and upper bounds of the 95%
confidence interval    """
    means = np.zeros(iterations)

    for i in range(iterations):
        bootstrap_sample = np.random.choice(data, size=len(data),
replace=True)
        means[i] = np.mean(bootstrap_sample)

    lower_bound = np.percentile(means, 2.5)
    upper_bound = np.percentile(means, 97.5)

    return (lower_bound, upper_bound)

# Example usage:
data = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
print(bootstrap_confidence_interval(data))

# (7.6, 14.6)
```

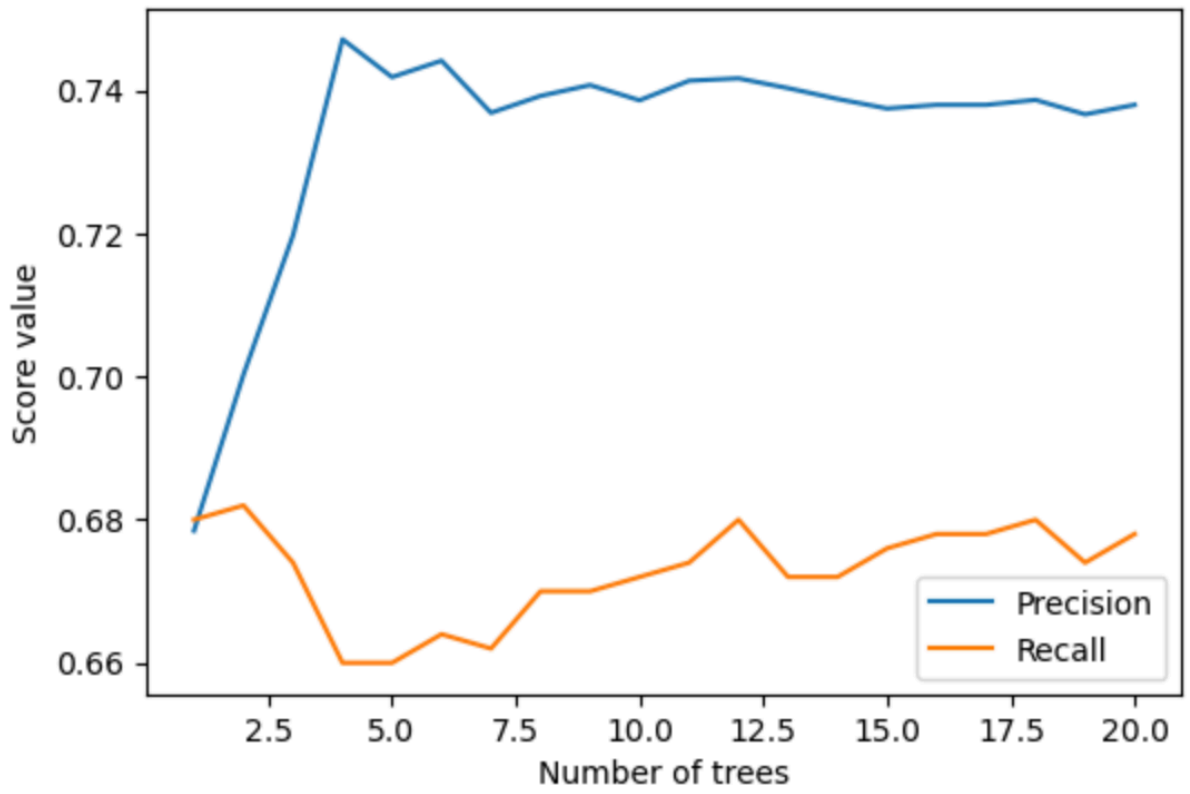
```
With this in mind, calculate:
1. If p = 0.1 and you have a change in log odds equals to +0.66, what will
be your new p?
2. If p = 0.9 and you have a change in log odds equals to +0.66, what will
be your new p?

def p_to_log_odds(p):
    return np.log(p/(1-p))
def log_odds_to_p(odds):
    return np.exp(odds) / (1+ np.exp(odds))
```

Question 3: Train a Random Forest model to predict the if a passenger of Titanic survived.

- Use random forest classifier with max tree depth of 3 (and random_state=0)
- Train the classifier by variating the number of trees from 1 to 20 (N)
- For each step estimate precision/recall with cross validation (10-folds)
- Plot 2 curves for different values of N

```
# Load the data
titanic = pd.read_excel('data/titanic.xls')
titanic_features = ['sex', 'age', 'sibsp', 'parch', 'fare']
X = pd.get_dummies(titanic[titanic_features])
X = X.fillna(X.mean())
y = titanic['survived']

from sklearn.ensemble import RandomForestClassifier

number_trees = [n for n in range(1, 21)]
precision_scores = []
recalls_scores = []


for nt in number_trees:
    clf = RandomForestClassifier(max_depth=3, random_state=0,
n_estimators=nt)
    clf.fit(X, y)
    precision = cross_val_score(clf, X, y, cv=10, scoring="precision")
    precision_scores.append(precision.mean())
    recall = cross_val_score(clf, X, y, cv=10, scoring="recall")
    recalls_scores.append(recall.mean())


fig, ax = plt.subplots(1, figsize=(6,4))

ax.plot(number_trees, precision_scores, label="Precision")
ax.plot(number_trees, recalls_scores, label="Recall")

ax.set_ylabel("Score value")
ax.set_xlabel("Number of trees")
ax.legend()
```

```
<matplotlib.legend.Legend at 0x7f97195d0a00>
```

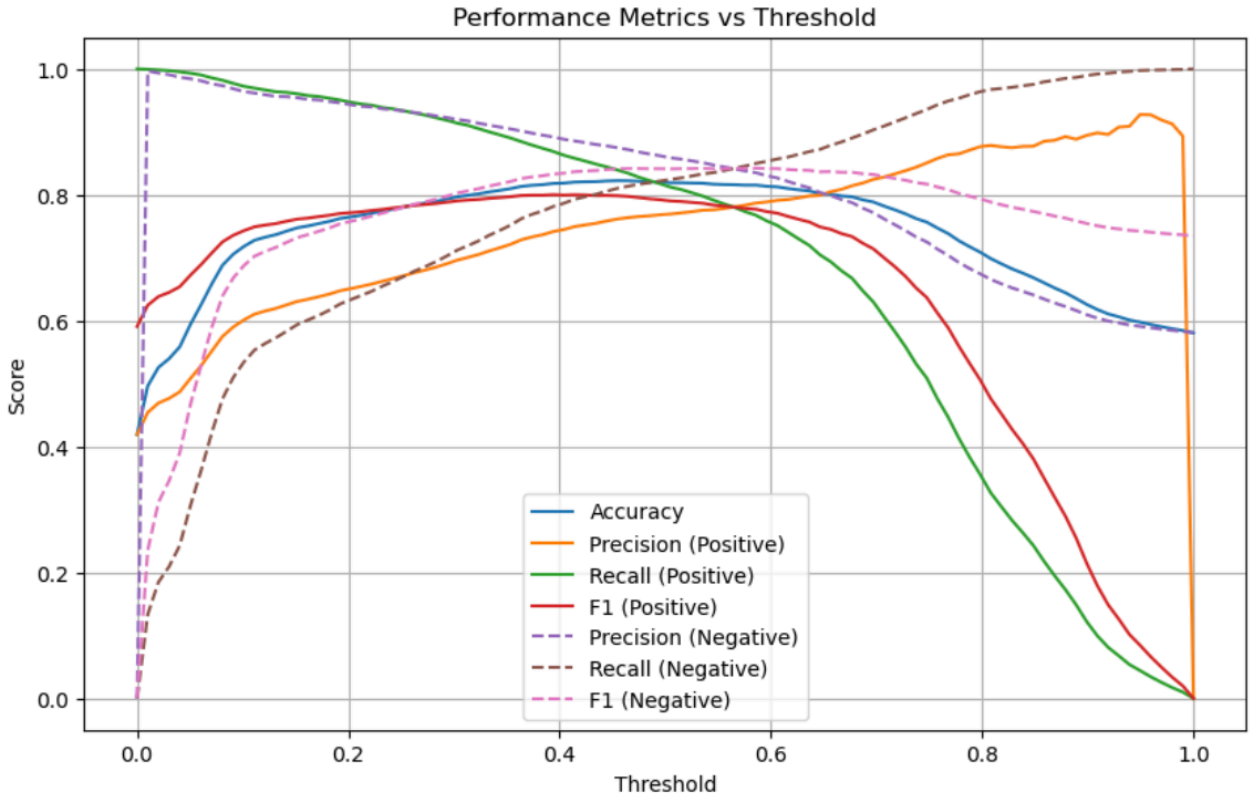


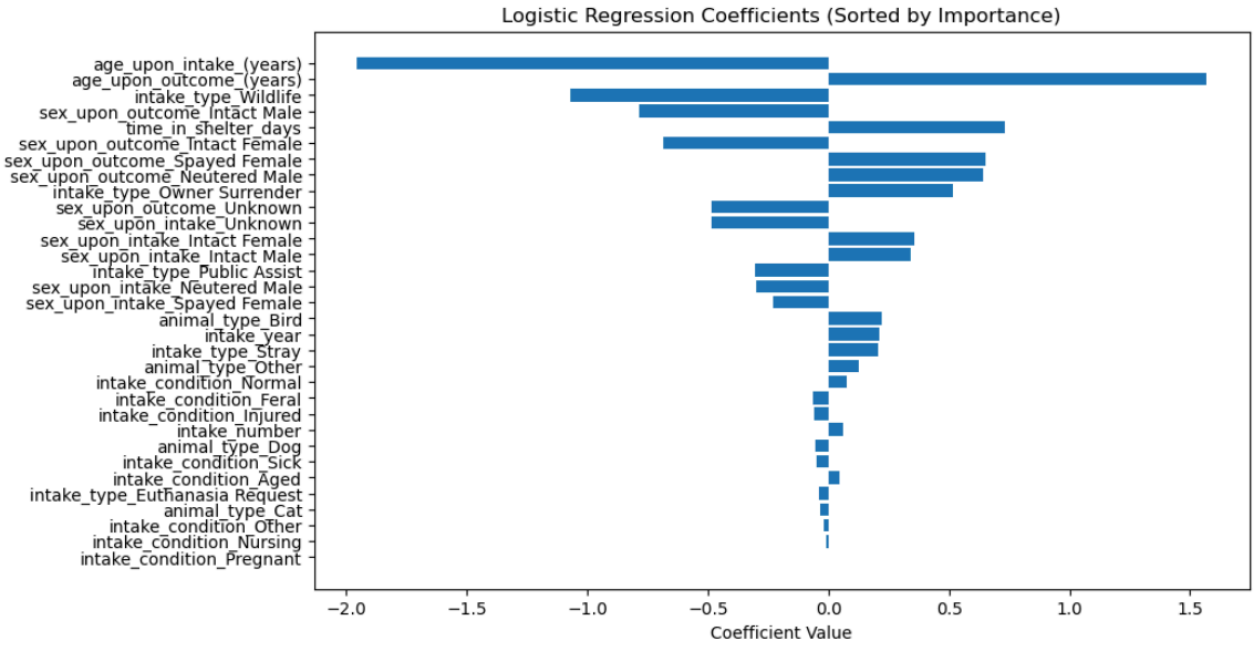

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score

# ------------------------------
# Load and clean data
# ------------------------------
columns = [
    'animal_type', 'intake_year', 'intake_condition', 'intake_number',
    'intake_type', 'sex_upon_intake', 'age_upon_intake_(years)',
    'time_in_shelter_days', 'sex_upon_outcome',
    'age_upon_outcome_(years)', 'outcome_type'
]

data = pd.read_csv('./data/aac_intakes_outcomes.csv', usecols=columns)
data.dropna(inplace=True)

# Binary label
data['adopted'] = (data['outcome_type'] == 'Adoption').astype(int)
data.drop('outcome_type', axis=1, inplace=True)

# ------------------------------
```

```python
# Dummy-variable encoding
# --------------------------------
categorical_cols = [
    'animal_type', 'intake_condition', 'intake_type',
    'sex_upon_intake', 'sex_upon_outcome'
]

data_encoded = pd.get_dummies(data, columns=categorical_cols)

X = data_encoded.drop('adopted', axis=1)
y = data_encoded['adopted']

# --------------------------------
# Train / test split (80 / 20)
# --------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# --------------------------------
# Standardization
# --------------------------------
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# --------------------------------
# Logistic Regression
# --------------------------------
model = LogisticRegression(max_iter=1000)
model.fit(X_train_std, y_train)

# Probabilities and binary predictions (threshold = 0.5)
y_prob = model.predict_proba(X_test_std)[:, 1]
y_pred = (y_prob >= 0.5).astype(int)

# --------------------------------
# Evaluation
# --------------------------------
cm = confusion_matrix(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
precision_pos = precision_score(y_test, y_pred, pos_label=1)
recall_pos = recall_score(y_test, y_pred, pos_label=1)
f1_pos = f1_score(y_test, y_pred, pos_label=1)

precision_neg = precision_score(y_test, y_pred, pos_label=0)
recall_neg = recall_score(y_test, y_pred, pos_label=0)
f1_neg = f1_score(y_test, y_pred, pos_label=0)
```

```
print("Confusion Matrix:")
print(cm)

print("\nMetrics for Positive Class (Adopted = 1)")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision_pos:.4f}")
print(f"Recall   : {recall_pos:.4f}")
print(f"F1-score : {f1_pos:.4f}")

print("\nMetrics for Negative Class (Not Adopted = 0)")
print(f"Precision: {precision_neg:.4f}")
print(f"Recall   : {recall_neg:.4f}")
print(f"F1-score : {f1_neg:.4f}")

# ============================================================
# C) Metrics as a function of threshold
# ============================================================
thresholds = np.linspace(0, 1, 100)

acc, prec_pos, rec_pos, f1_pos = [], [], [], []
prec_neg, rec_neg, f1_neg = [], [], []

for t in thresholds:
    y_hat = (y_prob >= t).astype(int)

    acc.append(accuracy_score(y_test, y_hat))
    prec_pos.append(precision_score(y_test, y_hat, pos_label=1,
zero_division=0))
    rec_pos.append(recall_score(y_test, y_hat, pos_label=1))
    f1_pos.append(f1_score(y_test, y_hat, pos_label=1))

    prec_neg.append(precision_score(y_test, y_hat, pos_label=0,
zero_division=0))
    rec_neg.append(recall_score(y_test, y_hat, pos_label=0))
    f1_neg.append(f1_score(y_test, y_hat, pos_label=0))

plt.figure(figsize=(10, 6))
plt.plot(thresholds, acc, label='Accuracy')
plt.plot(thresholds, prec_pos, label='Precision (Positive)')
plt.plot(thresholds, rec_pos, label='Recall (Positive)')
plt.plot(thresholds, f1_pos, label='F1 (Positive)')
plt.plot(thresholds, prec_neg, label='Precision (Negative)', linestyle='--
')
plt.plot(thresholds, rec_neg, label='Recall (Negative)', linestyle='--')
plt.plot(thresholds, f1_neg, label='F1 (Negative)', linestyle='--')

plt.xlabel("Threshold")
plt.ylabel("Score")
plt.title("Performance Metrics vs Threshold")
plt.legend()
```

```
plt.grid(True)
plt.show()

# ============================================================
# D) Logistic regression coefficients
# ============================================================
coef = model.coef_[0]
feature_names = X.columns

coef_df = pd.DataFrame({
    'feature': feature_names,
    'coefficient': coef,
    'abs_coef': np.abs(coef)
}).sort_values(by='abs_coef', ascending=False)

plt.figure(figsize=(10, 6))
plt.barh(coef_df['feature'], coef_df['coefficient'])
plt.gca().invert_yaxis()
plt.xlabel("Coefficient Value")
plt.title("Logistic Regression Coefficients (Sorted by Importance)")
plt.show()

'''
Confusion Matrix:
[[7618 1635]
 [1240 5440]]

Metrics for Positive Class (Adopted = 1)
Accuracy : 0.8196
Precision: 0.7689
Recall   : 0.8144
F1-score : 0.7910

Metrics for Negative Class (Not Adopted = 0)
Precision: 0.8600
Recall   : 0.8233
F1-score : 0.8413
'''
```



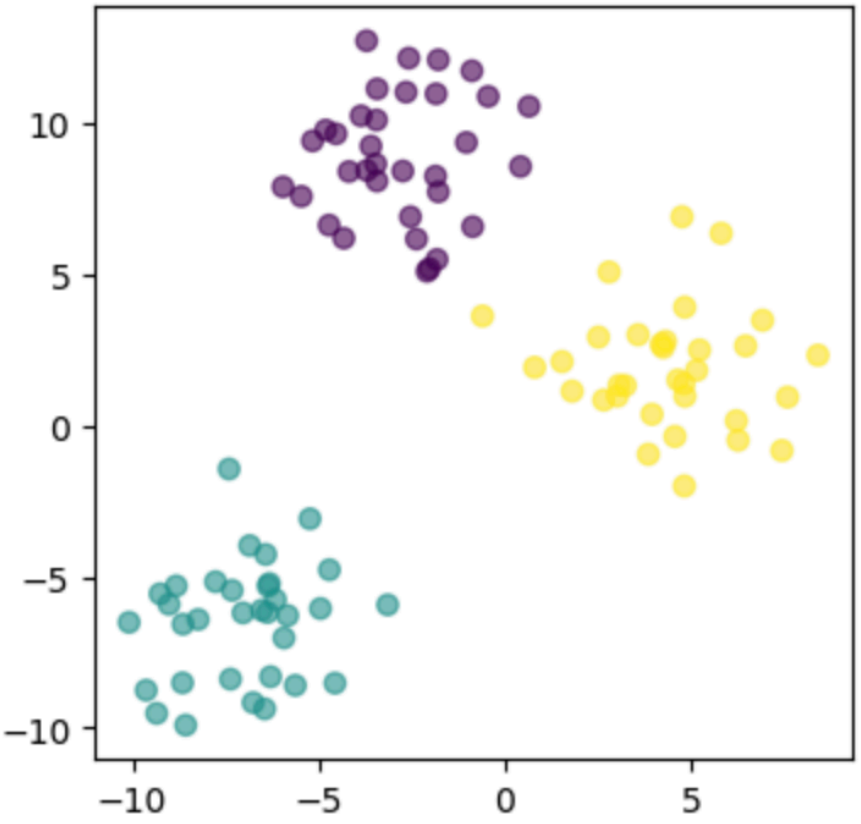


Clustering

```
import random
import math
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans, DBSCAN
```

```
from sklearn.metrics import silhouette_score
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

silhouettes = []

# Try multiple k
for k in range(2, 11):
    # Cluster the data and assigne the labels
    labels = KMeans(n_clusters=k, random_state=10).fit_predict(X)
    # Get the Silhouette score
    score = silhouette_score(X, labels)
    silhouettes.append({"k": k, "score": score})

# Convert to dataframe
silhouettes = pd.DataFrame(silhouettes)

# Plot the data
plt.plot(silhouettes.k, silhouettes.score)
plt.xlabel("K")
plt.ylabel("Silhouette score")


# For ELBOW method to choose K
def plot_sse(features_X, start=2, end=11):
    sse = []
    for k in range(start, end):
        # Assign the labels to the clusters
        kmeans = KMeans(n_clusters=k, random_state=10).fit(features_X)
        sse.append({"k": k, "sse": kmeans.inertia_})

    sse = pd.DataFrame(sse)
    # Plot the data
    plt.plot(sse.k, sse.sse)
    plt.xlabel("K")
    plt.ylabel("Sum of Squared Errors")

plot_sse(X)




fig, axs = plt.subplots(1, 1, figsize=(4,4), sharey=True)

# Plot the clusters with K = 3
labels = KMeans(n_clusters=3, random_state=0).fit_predict(X)
axs.scatter(X[:,0], X[:,1], c=labels, alpha=0.6)
```

```
<matplotlib.collections.PathCollection at 0x7fa68035afd0>
```

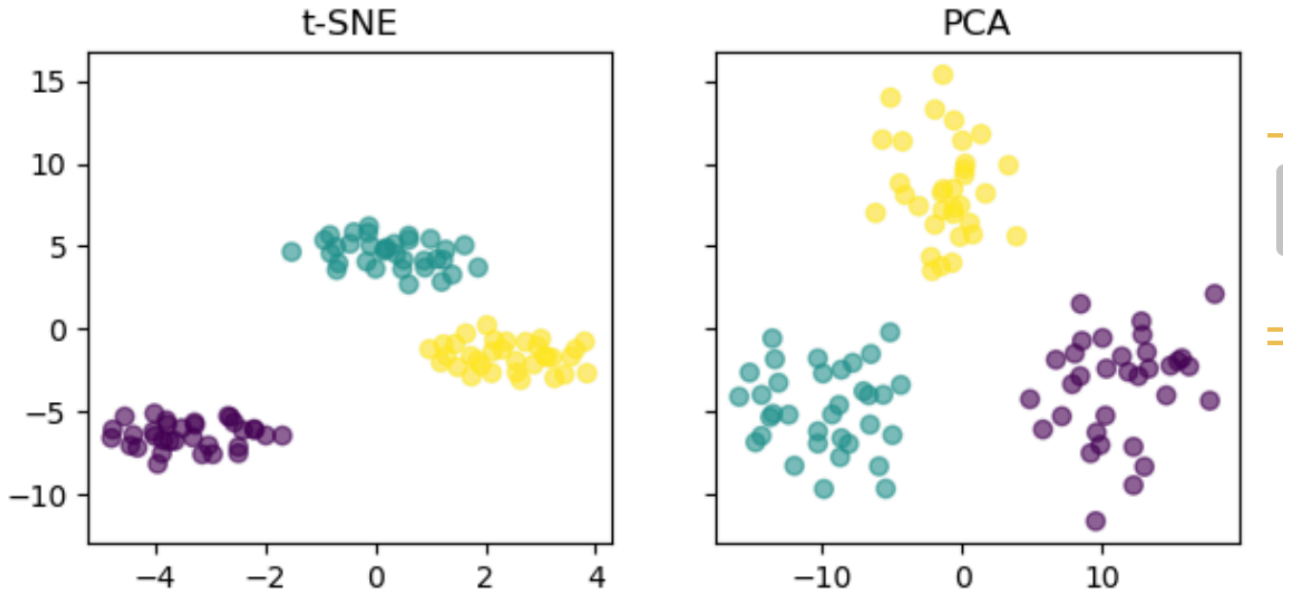


Visualizing higher dimension data

```
total_samples = 100

# This create some artifical clusters with standard dev. = 3
X10d, _ = make_blobs(n_samples=total_samples,
                             centers=top_secret_number,
                             cluster_std=3,
                             n_features=10,
                             random_state=0)

print("The features of the first sample are: %s" % X10d[0])
# The features of the first sample are: [ 7.05933272  4.20962197
-2.77357361  6.59147131 -6.64440614  9.47625342
 # -3.01996723  7.36384861  1.41157528  1.28459274]

# t-SNE
# PCA

X_reduced_tsne = TSNE(n_components=2, init='random', learning_rate='auto',
random_state=0).fit_transform(X10d)

print("The features of the first sample are: %s" % X_reduced_tsne[0])
```

```
# The features of the first sample are: [ 3.2180345 -1.7569892]


X_reduced_pca = PCA(n_components=2).fit(X10d).transform(X10d)

print("The features of the first sample are: %s" % X_reduced_pca[0])
# The features of the first sample are: [-6.17922102  7.01618025]

fig, axs = plt.subplots(1, 2, figsize=(7,3), sharey=True)

# Cluster the data in 3 groups
labels = KMeans(n_clusters=3, random_state=0).fit_predict(X10d)

# Plot the data reduced in 2d space with t-SNE
axs[0].scatter(X_reduced_tsne[:,0], X_reduced_tsne[:,1], c=labels,
alpha=0.6)
axs[0].set_title("t-SNE")

# Plot the data reduced in 2d space with PCA
axs[1].scatter(X_reduced_pca[:,0], X_reduced_pca[:,1], c=labels,
alpha=0.6)
axs[1].set_title("PCA")
```

```
Text(0.5, 1.0, 'PCA')
```



1.5 (9 pt)

You are concerned about potential “confounding” factors for your results in the previous question. Specifically, you would like to investigate the effect of “gender” on the outcomes.

[A, 3pt] Visualize the outcomes stratified by gender using a bar plot with standard deviation around the mean. Make sure you use clear x- and y-axis labels, a title, and a legend.

```
import matplotlib.pyplot as plt




df_meta_gp_gender = df_meta.groupby('gender')['outcome'].agg(['mean',
'std'])
df_meta_gp_gender.plot.bar(y='mean', yerr='std',
                           title="Mean Negotiation Outcomes Stratified by
Gender",
                           ylabel="negotiation outcome",
                           label="mean",
                          )
plt.show()
```

[B, 2pt] Perform a T-Test with a confidence interval of 0.95 to check if the outcomes based on gender are statistically significantly different. Print the resulting t-statistic, the p-value, and your interpretation of the result.

```
from scipy.stats import ttest_ind

male_outcomes = df_document[df_document['gender'] == 'male']['outcome']
female_outcomes = df_document[df_document['gender'] == 'female']
['outcome']

t_stat, p_value = ttest_ind(male_outcomes, female_outcomes,
equal_var=False)

print(f"t_stat: {t_stat}\np_value: {p_value}\n")

if p_value < 0.05:
    print(f'>There IS a statistically significant difference in
negotiation outcomes between genders')
else:
    print(f'>There is NOT a statistically significant difference in
negotiation outcomes between genders')
```

[C, 2pt] Finally, we would like to test if the gender distribution between high and low negotiation performers is significantly different. Please perform a chi-square test on the gender distribution of those negotiators scoring above the median outcome. Use a confidence interval of 0.95 and print the resulting chi-square statistic, p-value, and your interpretation of the result.

```
from scipy.stats import chi2_contingency

contingency_table = pd.crosstab(
    df_document['gender'],
    df_document['success']
)

chi2_stat, p_value, _, _ = chi2_contingency(contingency_table)

print(f"chi2_stat: {chi2_stat}\np_value: {p_value}\n")

if p_value < 0.05:
    print(f'>There IS a statistically significant difference in gender
distribution between high and low negotiation performers')
else:
    print(f'>There is NOT a statistically significant difference in gender
distribution between high and low negotiation performers')
```

```
def jaccard_similarity(list1, list2):
    s1 = set(list1)
    s2 = set(list2)
    return len(s1.intersection(s2)) / len(s1.union(s2))
```

```
import pandas as pd

def calculate_per_year(
    df,
    categories,
    col,
    year_start,
    year_end,
    what="count",
    keep_duplicates=True,
    duplicate_cols=None,
    cumulative=False,
):
    """
    Compute yearly statistics (count, mean, or sum) for a given column,
    filtered by category and year range.

    Parameters
    ----------
    df : pandas.DataFrame
        Input data containing at least 'channel_cat', 'upload_year', and
`col`.
    categories : list
        Categories to keep from df['channel_cat'].
```

```
    col : str
        Column on which the statistic is computed.
    year_start : int
        First year (inclusive).
    year_end : int
        Last year (inclusive).
    what : {'count', 'mean', 'sum'}, default 'count'
        Statistic to compute per year.
    keep_duplicates : bool, default True
        Whether to keep duplicate rows.
    duplicate_cols : list or None
        Columns used to identify duplicates (only if
keep_duplicates=False).
    cumulative : bool, default False
        Whether to return cumulative values over years.

    Returns
    -------
    pandas.DataFrame
        DataFrame indexed by year with the requested statistic.
    """

    # ------------------------------------------------------------------
    # 1. Filter rows by category and year range
    # ------------------------------------------------------------------
    df_filtered = df.loc[
        df["channel_cat"].isin(categories)
        & (df["upload_year"].between(year_start, year_end)),
        ["upload_year", col],
    ].sort_values("upload_year")

    # ------------------------------------------------------------------
    # 2. Optionally remove duplicate rows
    # ------------------------------------------------------------------
    if not keep_duplicates:
        df_filtered = df_filtered.drop_duplicates(
            subset=duplicate_cols, keep="first"
        )

    # ------------------------------------------------------------------
    # 3. Compute the yearly statistic
    # ------------------------------------------------------------------
    if what == "count":
        result_df = df_filtered.groupby("upload_year")[col].count()

        # Ensure all years in [year_start, year_end] are present
        all_years = pd.Series(
            0,
            index=range(year_start, year_end + 1),
            name=col,
```

```
        )
        result_df = result_df.add(all_years, fill_value=0)

    elif what == "mean":
        result_df = df_filtered.groupby("upload_year")[col].mean()

    elif what == "sum":
        result_df = df_filtered.groupby("upload_year")[col].sum()

    else:
        raise ValueError("`what` must be one of {'count', 'mean', 'sum'}")

    # ------------------------------------------------------------------
    # 4. Optionally compute cumulative values
    # ------------------------------------------------------------------
    if cumulative:
        result_df = result_df.fillna(0).cumsum()

    return result_df.to_frame(name=col)
```

Random sample from a df

```
# 3.3
BF = []

for i in range(200):
    df_sample = merged_df.sample(frac=1, replace=True)
    # A)
    y = (df_sample.VOT.values == 1).astype(int)
    y_pred = (df_sample.PP + df_sample.NN >= df_sample.PN).astype(int)
BF1 = roc_auc_score(y, y_pred)

    # B)
    y = (df_sample.VOT.values == 1).astype(int)
    y_pred = (df_sample.PP >= df_sample.PN).astype(int)    BF2 =
roc_auc_score(y, y_pred)        BF.append(BF1 - BF2)

print("95% CI:", np.quantile( np.array(BF), q=[0.025, 0.975]))
```